

pkgsentinel

AI 시대 공급망 악성 패키지 탐지 엔진 + Agentic Capability Manifest 제안

7팀 · 캡스톤 · 2026-05 · 피드백 요청용 요약

설치하지 않고 PyPI·npm 패키지를 7개 층으로 검사해 악성·위험을 탐지하는 오픈소스 엔진. AI 에이전트 패키지를 위한 **capability** 선언 사양을 함께 제안.

1. 다루는 문제

- 작년 npm Shai-Hulud 뮌: 신뢰받던 유명 패키지를 감염시켜 스스로 전파.
- 새 위험 ① AI가 없는 패키지 이름을 지어냄 → 공격자가 그 이름을 선점.
- 새 위험 ② AI 에이전트 패키지: 명령 실행·네트워크·비밀 읽기가 정상 기능 → 악성과 구분이 안 됨.
- 기존 도구(Trivy·Grype·OSV·Scanner)는 '이미 알려진 취약점' 매칭만 → 위 둘을 통과시킴.

2. 접근

원칙 — **Trust by Verification** (유명해도 봐주지 않는다)

과거 대형 사고(event-stream·ua-parser-js·XZ·Shai-Hulud)는 전부 유명 패키지에서 발생. 인기도는 검사 우선순위만 올리고, 판정 면제는 없음.

검출 백엔드 — 설치하지 않고 7개 층 (Layer 0-6)

- L0 알려진 악성인지 대조 (OSV·OSSF 22만+ 암호화 DB)
- L1 설치 없이 소스 추출 + 에이전트 패키지 판별
- L2 코드가 실제로 하는 행동을 순서대로 추출 (AST · tree-sitter)
- L3 알려진 공격 수법과 비교 (MITRE ATT&CK 568 TTP 임베딩 + 47-indicator + taint slicing)
- L4 의미 재검토 — Claude 3 에이전트 (semantic · diff · dependency 3축)
- L5 의존성 재귀 · 바이너리 분석 · 옵션 샌드박스
- L6 판정 + 표준 출력 (STIX · TAXII · Falco · CycloneDX VEX)

검증 layer — **Agentic Capability Manifest Diff** (직교)

- 7-layer 출력 (detected capability) ↔ 패키지 작성자 선언 (declared) 차이 계산
- 미선언 위험 capability 발견 시 verdict 격상 — 선언-기반 방어
- 7-layer 와 같은 단계가 아닌, 그 출력을 받는 직교 검증 layer (코드: agentic/classifier.py Step 2~4)

권위 있는 기반 위에 — 핵심 지식은 공인 자료 활용

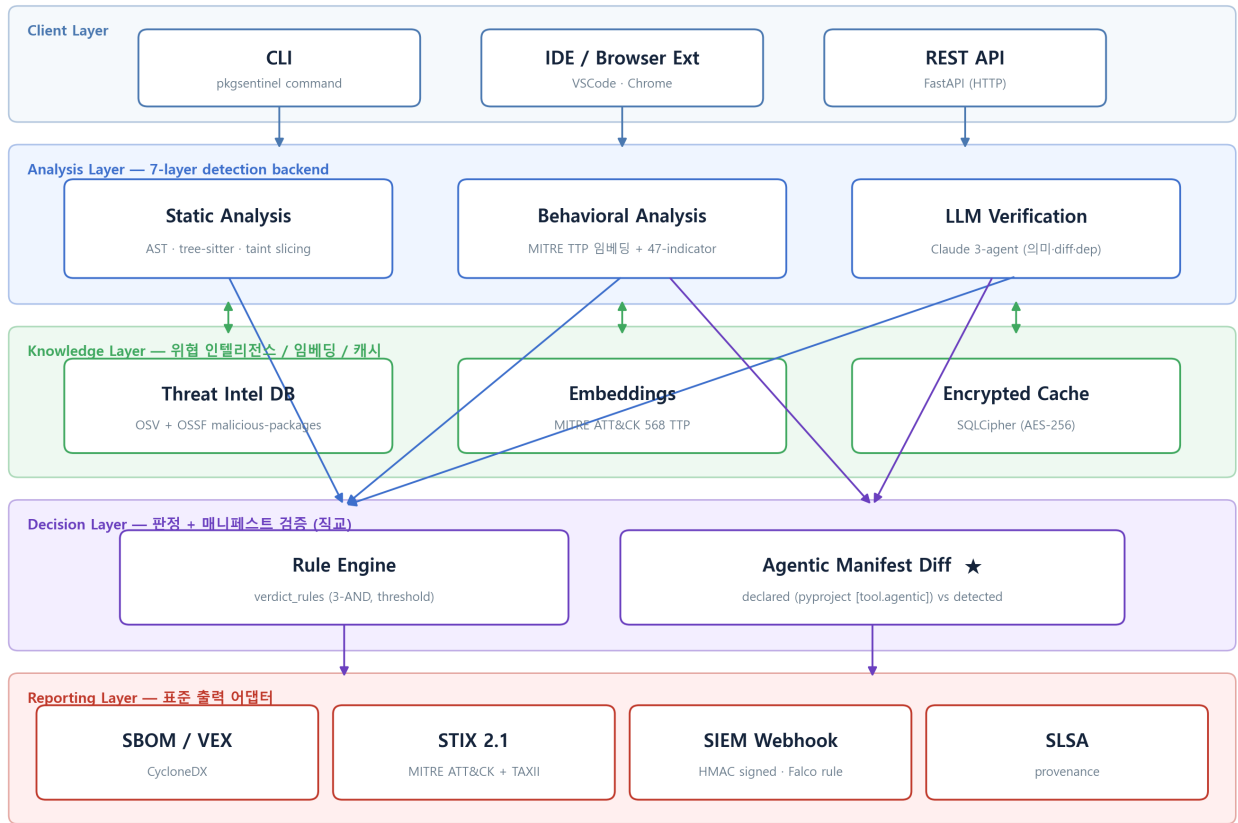
- 공격 분류: MITRE ATT&CK Enterprise 568 TTP (L3 임베딩 매칭)
- 악성 목록: OSV · OSSF malicious-packages (L0 lookup)
- 외부 평가: DataDog malicious-software-packages-dataset

3. 시스템 구조

입력 (CLI · Extension · HTTP API) → 7-layer detection backend (21-stage 구현) ↔ SQLCipher 암호화 DB · Claude 3-agent → Agentic Manifest 검증 layer (declared ↔ detected diff) → verdict_rules 최종 판정 → 표준 출력 sink.

pkgsentinel 시스템 구조 (5-layer)

역할 단위로 통일된 논리 계층 — 기술 스택은 박스 안 라벨로 부기



Client Analysis Knowledge Decision Reporting ★ Manifest layer (직교)

4. 주요 결과 (외부 검증 데이터셋)

DataDog malicious-software-packages-dataset 외부 검증 543개 (malicious 454 + benign 89). 자작 fixture 아님.

분석 완료 표본 (n=537, ERROR 6건 제외) recall 0.784 · F1 0.876 · precision 0.994

Shai-Hulud disjoint (162개) — advisory lookup 없이 행위 분석만으로 recall 1.000 ★

[npm 280 3-way 비교] OSV F1 0.926 / pkgSentinel F1 0.789 / Socket F1 0.962

↳ **registry (정상) FPR: OSV 14.8% / pkgSentinel 0% ★ / Socket 7.4%**

AI 다중 검증 추가 농침 +9%p 감소

※ ERROR 6건 (분석할 source 파일 자체 없음) 은 detection 능력 측정과 무관 → 분모 제외 (1a 정정). 이전 산정 방식 (ERROR 를 FN 으로 카운트) 은 recall 0.773.

※ Shai-Hulud disjoint 162개 = OSV/OSSF advisory 의 Shai-Hulud 영향 203개 중 기존 fixture (54개) 와 disjoint 한 187개 중 archive 확보 가능한 86.6% (DataDog 2025-11-24 신규 업데이트 활용). pkgSentinel 은 stage 01 (OSV lookup) 호출 없이 순수 행위 분석 (AST + 47-indicator + TTP 임베딩 + taint slicing) 만으로 162개 100% 탐지 — Shai-Hulud worm 의 동일 payload 패턴을 advisory 매칭 없이도 추출함을 입증.

※ benign 89개에 정상 agentic (LangChain-CrewAI-MCP) 0건 — precision 0.994 는 agentic 부재 dataset 편향에 의존할 가능성. 정상 agentic 100개에 대한 manifest ON/OFF FPR 별도 측정 예정 (매니페스트 가치 직접 정량).

※ 1c (weak TTP 누적 정책) simulation 결과 모든 setting 에서 precision 폭락 (-2~-7%p) — 정상 메이저 npm 패키지 (react, lodash, webpack, eslint 등) 가 자연스러운 TTP 누적으로 새 FP. recall 추가 향상은 매니페스트 검증 layer 도입 (architectural) 으로 풀어야 함이 정량 확인.

5. 표준 제안 — Agentic Capability Manifest

AI 에이전트 관련 패키지는 명령 실행·네트워크·비밀 접근이 정상 기능이라 기존 정적 분석으로 악성과 구분이 어렵다. 이를 위해 '이 패키지가 어떤 위험 capability를 쓸지 설치 전에 선언'하게 하는 매니페스트를 제안. 본 도구의 7-layer 검출 백엔드와 직교(orthogonal)하는 검증 layer로서, 탐지된 capability와 작성자 선언의 차이를 계산해 미선언 위험을 잡는다.

키 pyproject.toml / package.json 의 [tool.agentic] 섹션
검증 declared (선언) ↔ detected (실제 행동) diff → 미선언 위험 탐지
포지셔닝 보안 통제가 아닌 '투명성 관례' / 워크숍 페이지·RFC 트랙

선행연구 대비 빈칸

- Deno permissions: 런타임 · Deno 한정 (설치 전 / 생태계 무관 X)
- Capslock (Google, Go): capability 추론 (작성자 선언 아님, Go 한정)
- CycloneDX / SBOM: 구성요소 목록 (capability 선언 아님)
- → '범용 생태계 · 작성자 선언형 · 설치-전 기계검증 가능한 agentic capability manifest'는 비어 있음

6. 산출물

- 오픈소스 엔진 (Python, 80+ 모듈, 404개 자동 테스트 통과, CI/CD)
- Agentic Capability Manifest 사양 + 레퍼런스 구현 + 자동 생성기
- 외부 검증셋 평가 결과 (4-cell: stub × claude × ON/OFF, n=543)
- 표준 출력 어댑터: CycloneDX VEX · STIX 2.1 / TAXII · Falco · HMAC-signed Webhook
- 데모 도구 (보존된 악성 아카이브 분석 — Shai-Hulud 등 unpublish된 샘플 시연용)

7. 피드백 부탁드립니다 싶은 지점

- 추가로 측정해 두면 좋을 평가 지표 / 비교 대상
 - 비슷한 소프트웨어와의 차별점이라고 생각되는 부분이 '위험 capability를 설치 전 기계검증 가능하게 선언하고, 선언된 부분과 비교해 미선언된 부분을 잡는다' 라는 구조에서 나온다고 생각합니다. 이 부분을 설득력 있게 어필할 수 있을까요?

8. 참고 자료

★ = 본 프로젝트에 직접 반영된 자료

8.1 학술 논문 (주요)

- ★ Unveiling Malicious Logic: Statement-Level Taxonomy for Python Packages (2025, arXiv:2512.12559) — 47개 지표 텍소노미 → V2 47-indicator 매처
- ★ Cerebro: Single Model of Malicious Behavior Sequence for NPM/PyPI (ACM TOSEM 2025, arXiv:2309.02637) — Behavior Sequence + tree-sitter
- DONAPI: Behavior Sequence Knowledge Mapping (USENIX Security 2024, arXiv:2403.08334) — 132 API 카탈로그
- Taint-Based Code Slicing for LLMs (2025, arXiv:2512.12313) — V2 Stage 15 taint slicer 근거
- Mind the Gap: LLMs for Malicious Package Detection (2025, arXiv:2602.16304) — 13개 LLM 비교
- LAMPS: LLM Multi-Agent for PyPI (2025, arXiv:2601.12148) — V2 Stage 16 다중 에이전트
- NPM Benchmark: Empirical Analysis (2025, arXiv:2603.27549) — 6420 mal + 7288 benign
- Robust Detection in Industry Environments (2024, arXiv:2409.09356) — FP 율 튜닝 방법론
- We Have a Package for You: Package Hallucinations by LLMs (USENIX Security 2024) — 19.7% 환각률

8.2 학술 논문 (LLM 환각 · 공급망 보조)

- Library Hallucinations in LLMs: Risk Analysis Grounded in Developer Queries (2025, arXiv:2509.22202)
- Malicious Package Detection using Metadata Information (ACM Web Conf 2024, arXiv:2402.07444)
 - Cutting the Gordian Knot: Detecting Malicious PyPI Packages via a Knowledge-Mining Framework (USENIX Security 26, arXiv:2601.16463)
- An Empirical Study of Malicious Code in PyPI Ecosystem (ASE 2023, arXiv:2309.11021)

- Supply-Chain Poisoning Attacks Against LLM Coding Agent Skill Ecosystems (2026, arXiv:2604.03081)
- Importing Phantoms: Measuring LLM Package Hallucination Vulnerabilities (2025, arXiv:2501.19012)
- One Detector Fits All: Robust and Adaptive Detection of Malicious Packages from PyPI to Enterprises (ACSAC 2025, arXiv:2512.04338)

8.3 공식 표준 / 프레임워크

- ★ NIST SP 800-218 — Secure Software Development Framework v1.1 (csrc.nist.gov/projects/ssdf)
- ★ MITRE ATT&CK — Enterprise Matrix (attack.mitre.org / github.com/mitre/cti)
- ★ SLSA v1.0 — Supply-chain Levels for Software Artifacts (slsa.dev, OpenSSF)
- ★ OpenSSF Scorecard — 18+ 보안 자동 평가 (scorecard.dev)
- ★ CycloneDX (OWASP) — SBOM / VEX 표준 (출력 포맷)

8.4 데이터 소스

- ★ MITRE ATT&CK Enterprise STIX 2.x JSON — 568 techniques (임베딩 매칭)
- ★ OSV (Google) — Open Source Vulnerabilities (osv.dev): PyPI 11,164 · npm 212,465 advisory 로컬 캐시
- ★ GitHub Advisory Database (GHSA) — github.com/advisories
- ★ PyPI JSON API — pypi.org/pypi/{pkg}/json (메타데이터 게이트)
- ★ npm Registry — registry.npmjs.org/{pkg} (메타데이터 게이트)
- ★ DataDog malicious-software-packages-dataset — 외부 평가셋 (n=543 본 평가에 사용)

8.5 사용 도구 / 오픈소스 라이브러리

- ★ tree-sitter (+ tree-sitter-javascript) — JS/TS AST 파싱
- ★ sentence-transformers — 'all-MiniLM-L6-v2' 임베딩 모델 (TTP 매칭)
- ★ Anthropic Claude API — LLM 다중 에이전트 검증
- ★ SQLCipher (sqlcipher3) — AES-256 페이지 암호화 DB
- ★ Docker — 샌드박스 격리 컨테이너
- ★ pefile — Windows PE 바이너리 import 분석
- ★ pyelftools — Linux ELF 심볼 분석
- ★ rapidfuzz — 편집거리 (typosquat 후보 생성)
- FastAPI — HTTP API 서버

8.6 관련 / 경쟁 프로젝트 (비교 검토)

- Phantom Guard (OSS) — 의존성 파일 기반, Python 비동기
- SlopGuard (OSS) — AI 환각 패키지 메타데이터 중심
- Socket.dev (상용) — 메타+소스+AI, 70+ 행위 신호, AI 의심 시 LLM 심층 평가
- GuardDog (DataDog, OSS) — PyPI/npm 악성 탐지 Semgrep+YARA+metadata (DataDog 데이터셋 제작사)
- OSSF Package Analysis (OSS) — gVisor 동적 샌드박스 + 시간에 따른 행위 변화 추적
- Phylum (now Veracode, 상용) — 패키지 행위 위험 점수
- ★ Capslock (Google, Go) — capability 추론 (manifest 비교 시 인용)
- ★ Deno permissions — 런타임 허용 명령 (manifest 비교 시 인용)

전체 카탈로그 (54건, BibTeX 포함): docs/references/00_MASTER_INDEX.md 및 01-06 파일 참조.